

ROTEIRO PRÁTICO

Pandas + CSV em Python

Passo 1 — Instalar o Pandas

Rode esse comando uma vez no terminal antes de tudo. O Pandas não vem instalado por padrão no Python.

TERMINAL / CMD

```
pip install pandas
```

Dica: Se aparecer 'Successfully installed pandas', está pronto para seguir!

Passo 2 — Criar um arquivo CSV de exemplo

Salve esse conteúdo como alunos.csv na mesma pasta do seu script Python. Você vai usar esse arquivo nos próximos passos.

ALUNOS.CSV

```
nome,idade,nota
Ana,19,8.5
Bruno,20,7.0
Carla,18,9.2
Diego,21,6.8
```

Dica: CSV é um arquivo de texto simples onde cada linha é uma linha da tabela, e os valores são separados por vírgula.

Passo 3 — Ler o CSV com Pandas

Aqui começa a mágica. A função `read_csv()` lê o arquivo e transforma em um `DataFrame`, que é como uma tabela em Python.

PYTHON

```
import pandas as pd

# le o arquivo CSV
df = pd.read_csv('alunos.csv')

# exibe a tabela completa
print(df)
```

SAÍDA

	nome	idade	nota
0	Ana	19	8.5
1	Bruno	20	7.0
2	Carla	18	9.2
3	Diego	21	6.8

Dica: O 'df' é só um nome — abreviação de `DataFrame`. Você pode chamar de qualquer coisa, como `tabela` ou `dados`.

Passo 4 — Explorar os dados

Esses são os comandos que você vai usar toda vez que abrir um CSV novo. Eles ajudam a entender a estrutura do arquivo rapidinho.

PYTHON

```
# primeiras 3 linhas
print(df.head(3))
```

```
# quantas linhas e colunas tem?
print(df.shape)      # (4, 3)

# nomes das colunas
print(df.columns)

# resumo geral do arquivo
print(df.info())
```

Passo 5 — Acessar colunas e fazer cálculos

Use o nome da coluna entre colchetes para acessar só aquela coluna. A partir daí você já consegue fazer cálculos simples.

PYTHON

```
# ver só a coluna 'nota'
print(df['nota'])

# media das notas
print(df['nota'].mean())  # 7.875

# maior e menor nota
print(df['nota'].max())    # 9.2
print(df['nota'].min())    # 6.8
```

Passo 6 — Filtrar linhas

Quer ver só os alunos com nota acima de 8? Usar uma condição dentro dos colchetes — é quase como falar português.

PYTHON

```
# alunos com nota maior que 8
```

```
aprovados = df[df['nota'] > 8]
print(aprovados)
```

SAÍDA

	nome	idade	nota
0	Ana	19	8.5
2	Carla	18	9.2

Passo 7 — Salvar resultado em novo CSV

Depois de filtrar ou modificar os dados, você pode salvar o resultado em um novo arquivo CSV para usar depois.

PYTHON

```
# salva sem o numero de indice na primeira coluna
aprovados.to_csv('aprovados.csv', index=False)
```

Dica: O `index=False` evita que o Pandas salve a numeração das linhas (0, 1, 2...) como uma coluna extra no arquivo.

Resumão — Métodos mais usados

Cole esse bloco no seu caderno. São os comandos que você vai usar em 90% dos projetos com dados:

PYTHON

```
import pandas as pd

df = pd.read_csv('arquivo.csv') # abre o arquivo
df.head(5)                      # primeiras 5 linhas
df.shape                       # (linhas, colunas)
df.info()                      # tipos e valores nulos
```

<code>df.columns</code>	<code># nomes das colunas</code>
<code>df['coluna'].mean()</code>	<code># media</code>
<code>df['coluna'].max()</code>	<code># maior valor</code>
<code>df['coluna'].min()</code>	<code># menor valor</code>
<code>df[df['coluna'] > valor]</code>	<code># filtro</code>
<code>df.to_csv('novo.csv', index=False)</code>	<code># salvar</code>